YUKTISHAALAA AI LAB

YuktAI

Web apps everyone can use, in Indian languages.

Complete documentation: the six packages, every option, the AI inside, accessibility, privacy, testing, publishing and the full release history up to 6.0.3 / 6.0.4.

Package	Role	Version in repository
`@yuktishaalaa/yuktai`	YuktAI Toolkit	6.0.3
`@yuktishaalaa/assistant`	YuktAI Assistant	6.0.4
`@yuktishaalaa/a11y`	Accessibility engine	6.0.3
`@yuktishaalaa/ask-page`	Ask engine	1.0.2
`create-yuktai`	App starter	1.0.0
`yuktai`	Command line	1.4.0

Author: Sandeep Miriyala · License: GPL-3.0-only · Docs site: sandeep-miriyala.github.io/yuktai
Up to date as of 14 November 2026 (repository commit f133b30).

Contents

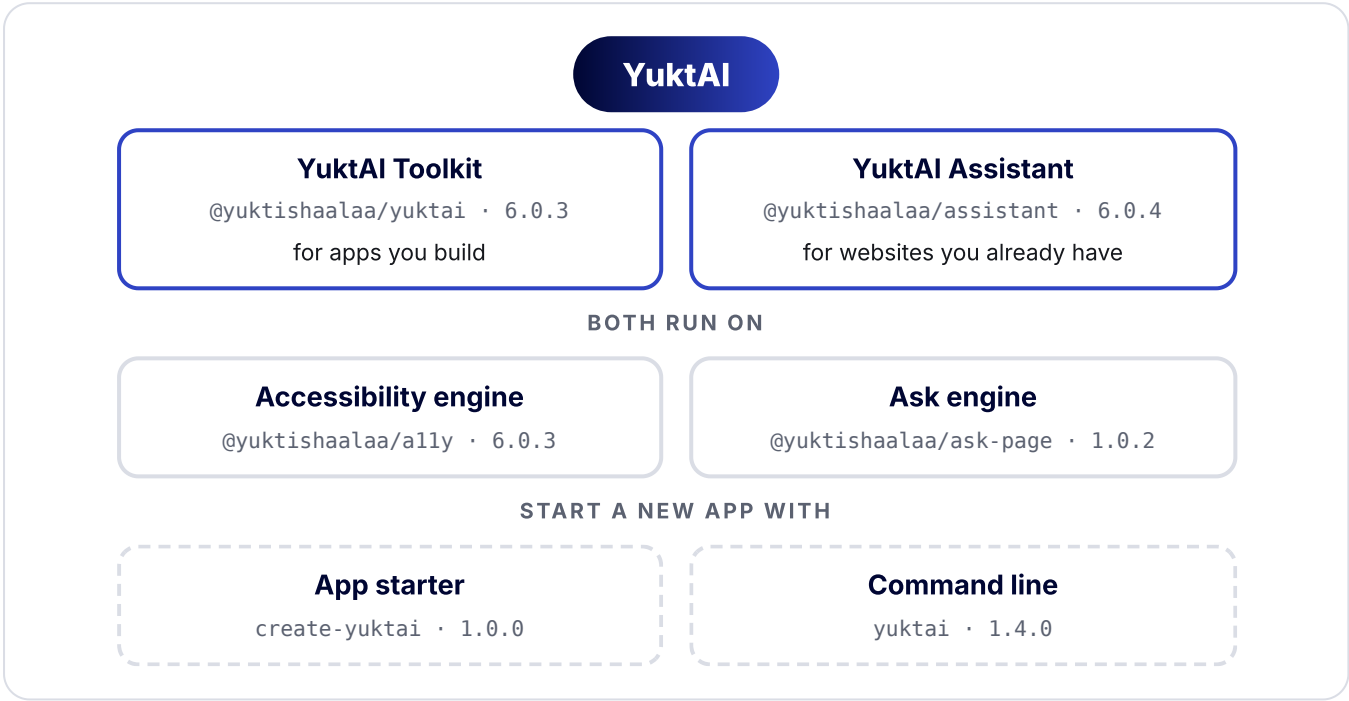
1. What YuktAI is
2. Quick start: four ways in
3. YuktAI Assistant
4. Accessibility engine (a11y)
5. Ask this page (ask-page)
6. YuktAI Toolkit
7. Starters, CLI and generated apps
8. AI concepts used in YuktAI
9. AI concepts in Telugu (సారాంశం)
10. Privacy and safety
11. Architecture and repository
12. Testing and quality
13. Releasing: GitHub, npm, website
14. Release history
15. Known limits and roadmap
16. Glossary

> **Versions.** This document describes the code in the repository at commit `f133b30` . On npm at the time of writing: yuktai 6.0.2, a11y 6.0.2, assistant 6.0.3, ask-page 1.0.2. The repository versions yuktai 6.0.3, a11y 6.0.3 and assistant 6.0.4 are ready and wait for `npm publish` + approval (chapter 13).

1. What YuktAI is

YuktAI is a free, open-source toolkit for web apps everyone can use, in Indian languages. It brings WCAG 2.2 accessibility, a Rust-fast data grid for 10 lakh rows and AI that answers from your own data to Next.js, React and plain HTML. Everything runs in the visitor's browser: no server, no API key, nothing sent anywhere.

Six packages, one family



If you want to...	Use
Build a new accessible app (grid, icons, UI components, agents)	**YuktAI Toolkit** `@yuktishaalaa/yuktai`, or start with `npm create yuktai@latest`
Add accessibility and "Ask this page" to an existing website	**YuktAI Assistant**: one script, npm, or the browser extension
Only the accessibility engine, from one HTML attribute	`@yuktishaalaa/a11y`
Only the Ask panel or its search as a library	`@yuktishaalaa/ask-page`

The principles behind every package

- **Free and open source.** GPL-3.0-only. No paid tier.
- **Visitor data is never taken.** No tracking, no analytics. What visitors type, passwords and codes are never read. Preferences stay on the device.
- **Do no harm.** Accessibility fixes are only made where they are safe; everything is undoable; "not checked" is never reported as "pass".
- **On-device AI first, honest without it.** AI is used only when it is already on the device; without it the features still work and say what they are (for example, "search, no AI in this browser").

- **Indian languages.** Telugu, Hindi and English interfaces; the language of every part of a page is found from its script; voices are picked per language.

2. Quick start: four ways in

A. Any website: one script (YuktAI Assistant)

```
<script>
  window.YuktaiAssistantConfig = {
    features: ["ask", "a11y"],
    level: "AA",
    title: "My site help",       // your panel name
    color: "#B3261E"             // your brand colour
  };
</script>
<script src="https://cdn.jsdelivr.net/npm/@yuktishaalaa/assistant@6/dist/assistant.global.js"
defer></script>
```

B. Next.js (App Router)

```
// app/layout.tsx
import { YuktaiAssistant } from "@yuktishaalaa/assistant/next"

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (<html><body>{children}
    <YuktaiAssistant features={["ask", "a11y"]} level="AA" title="My site help" color="#B3261E" />
  </body></html>)
}
```

C. A new app in one command

```
npm create yuktai@latest my-app
cd my-app
npm run dev        # site + Rust build + Python API together, http://localhost:3000
```

D. For everyone: the browser extension

Download `yuktai-assistant-chrome-edge-6.0.4.zip` from the docs site (YuktAI Assistant page), unzip it, open `chrome://extensions`, turn on **Developer mode** and choose **Load unpacked**. Works in Chrome, Edge and Brave on desktop. **Alt+Shift+Y** opens it, **Alt+Shift+A** asks the page.

Only the accessibility engine

```
<body data-yuktai-a11y="auto" data-yuktai-level="AA">
  …
  <script src="https://cdn.jsdelivr.net/npm/@yuktishaalaa/a11y@6/dist/a11y.global.js" defer>
</script>
</body>
```

3. YuktAI Assistant `@yuktishaalaa/assistant 6.0.4`

Web accessibility and Ask this page, in one panel. Built on the a11y and ask-page engines. A custom element `<yuktai-assistant>` with an open shadow root, marked `data-yuktai-ui` so the accessibility engine never "fixes" it.

What visitors get

Tab	Contents
Accessibility (అందుబాటు / सुलभता)	Go to main content · WCAG level Off/AA/AAA · fix low contrast · reduce motion · clear focus ring · wider spacing · page language (found automatically) · read the page aloud · panel language (English, తెలుగు, हिन्दी) · report: fixed / check / unknown, with a **"What to check"** list (message, WCAG number, where)
Colours	Default, High contrast or Soft theme · smart dark mode Off / On / Device · text size 100–200%. Saved on the device only.
Ask	Ask this page: answers from the page itself, with the source highlighted.

Keyboard: tabs with arrow keys, Escape closes and returns focus, **Alt+Shift+Y** opens and closes the panel on websites. The panel follows the visitor's dark or high-contrast choice, shows ✓ and outlines in Windows High Contrast, and its `lang` matches the words it shows.

All options

Option	Values	Default
`features`	`"a11y"`, `"ask"`	both
`level`	`"AA"`, `"AAA"`, `"off"` (`"A"` is turned into AA with a console note)	`"AA"`
`contrast`	`"check"`, `"fix"` (darken faint text)	`"check"`
`mode`	`"auto"` (safe fixes), `"check"` (report only)	`"auto"`
`title`	panel name, e.g. your site's name (max 60 characters)	"YuktAI Assistant"
`color`	brand colour; text on it is picked for contrast	#2F43C4
`side`	`"right"`, `"left"` (Ask opens on the same side)	`"right"`
`offset`	px from the corner: `90` or `{ x: 20, y: 90 }`	20
`panelLang`	`"auto"`, `"en"`, `"te"`, `"hi"` (visitors can change it)	`"auto"`
`shortcut`	Alt+Shift+Y	`true`
`launcher`	`false`: no ♿ button, open it from your own button	`true`
`ask`	Ask this page options (`modes`, `exclude`, `webmcp` …, chapter 5)	
`storageKey`	localStorage key for the visitor's choices	`"yuktai-assistant"`

> A typo or a wrong value (`titel`, `level: "A"`, `color: "blu"`, an unknown feature) is reported in the browser console and the default is used, so the panel never breaks. `checkConfig()` is exported for your own checks.

React

```
import { useEffect } from "react"
import { mountAssistant } from "@yuktishaalaa/assistant"

useEffect(() => {
  const assistant = mountAssistant({ features: ["ask", "a11y"], level: "AA" })
  return () => assistant.destroy()       // removes the panel and every change
}, [])
```

Browsers

npm and script: all modern browsers on desktop and phones. Read aloud uses the device's voices; on-device AI answers in Ask need Chrome (other browsers get page search with highlights). Extension: Chrome, Edge, Brave on desktop.

4. Accessibility engine `@yuktishaalaa/a11y 6.0.3`

Web accessibility for any page, in the browser: safe ARIA, WCAG AA/AAA contrast, `lang` for every part found automatically, read aloud in the right voice, visitor preferences and smart dark mode. One attribute, no server.

Attributes (no code)

Attribute	Values	Default
`data-yuktai-a11y`	`auto` (safe fixes + report) · `check` (report only) · `off`	`auto`
`data-yuktai-level`	`AA` · `AAA` · `off`	`AA`
`data-yuktai-contrast`	`check` · `fix`	`check`
`data-yuktai-lang`	`auto` · `off`	`auto`
`data-yuktai-speak`	`on` · `off` (🔊 button after h1–h3)	`off`

Any element opts out with `data-yuktai-a11y="off"`.

npm

```
import { createA11y } from "@yuktishaalaa/a11y"

const a11y = createA11y({ level: "AA", contrast: "check" })
const report = a11y.run()                          // { fixed, check, unknown }
a11y.setPrefs({ textSize: 130, dark: "device" })
a11y.destroy()                                     // removes every change
```

Do no harm

- **Native elements are checked, never relabelled.** No made-up "Button" or "Image" labels, no `alt=""` on pictures that may matter. A wrong label is worse than none.
- **Your `lang` and ARIA are never changed.** Only a missing `lang` is added.
- **The only structural fix:** an element with an inline `onclick` that is not a control and has a name gets `role="button"`, `tabindex="0"` and Enter/Space.
- **"Not checked" is never "pass".** Text over images, closed web components and cross-site iframes are reported as *unknown*.
- **Everything is undoable** (`undo()` / `destroy()`): each change carries a `data-yuktai-*` mark.

Report

Three groups: **fixed** (changed safely, undoable), **check** (a person should look; the message says how), **unknown** (could not be checked). Each finding has its WCAG 2.2 criterion, a short path and messages in English and Telugu. Checks include contrast (1.4.3 / 1.4.6), missing language (3.1.1 / 3.1.2), controls named only by symbols or emoji (2.4.6) and broken "Skip to main content" links (2.4.1).

Language, automatically

The script of the text tells the language in every browser: Telugu, Tamil, Kannada, Malayalam, Gujarati, Punjabi, Odia, Bengali, Devanagari, Arabic, Latin and more. Short English words inside a Telugu or Hindi sentence ("Login", "PDF") keep the sentence's language. Chrome's `LanguageDetector` refines scripts

shared by several languages. Single-page apps are re-checked when the page changes; open shadow roots get `lang` too.

Read aloud

`speak()` uses the browser's Web Speech. Text is cut where the language changes; each part is read with a voice for its language (`te-IN` , `en-IN` ...). Emoji, icon-font characters, arrows and ★ ✓ are skipped; separators become pauses; dates with "/" and ₹ are kept. `readTarget()` reads `<main>` , else the largest `<article>` , else the page without menus and footer. Hidden content is never read.

Visitor preferences and smart dark mode

`setPrefs({ textSize, spacing, reduceMotion, focusRing, theme, custom, dark })` . Text size uses CSS `zoom` , so sites sized in px grow too. **Smart dark mode** (since 6.0.2) changes each colour on its own instead of inverting the page:

- Backgrounds darker, text and icons lighter, hue kept (a blue button stays blue).
- Every text colour is checked against its new background and moved until it passes the visitor's level (AA 4.5:1, AAA 7:1).
- Photos, videos, emoji, maps and iframes keep their colours; text on background pictures keeps its colour; gradients become one dark colour; gradient text keeps its gradient, lightened (6.0.3).
- CSS-drawn icons (`::before` / `::after`) are recoloured too (6.0.3).
- Pages that are already dark are left alone; new content is darkened as it arrives; "Device" follows the operating system live; off restores the page exactly.

5. Ask this page `@yuktishaalaa/ask-page 1.0.2`

A side panel that answers questions from the page itself, with the source highlighted. Rust/WebAssembly search, on-device AI, read-only tools for browser agents (WebMCP). Nothing is sent anywhere.

Three modes

Mode	Search	Answer	Download
⚡ Fast	Rust BM25 (keywords)	Best sentences from the page, quoted	~100 KB
⚖️ Balanced	BM25 + multilingual embeddings, merged with RRF	Chrome built-in AI if ready, else quoted	~118 MB once, after the visitor agrees
🎯 Best	Same	Chrome built-in AI, else WebLLM (if the owner sets `webllmUrl`)	More, once

How a question is answered

question → page-structure shortcut (summary, forms, images, a11y) → retrieve: BM25 [+ int8 vectors] → RRF → confidence check → weak: "not on this page" / strong: answer → cite + highlight

Part	Technology
Panel	Web component, Shadow DOM
Reader + privacy filter	Live DOM walk (so sources can be highlighted); form values never read
Search core	Rust → WebAssembly: Telugu-aware tokenizer, BM25, int8 vectors, RRF, confidence. TypeScript twin when WebAssembly is blocked (strict CSP)
Question words	Telugu and Hindi question words (ఎన్ని, అంటే, कितने…) are removed before keyword search (1.0.1)
Embeddings	Transformers.js v4 + `Xenova/multilingual-e5-small`
Answers	Chrome Prompt API → WebLLM → quoted passage; Translator and Summarizer APIs
Flow	Five plain functions; the same as a LangGraph.js `StateGraph` in `@yuktishaalaa/ask-page/langgraph`
Threads	Web Worker (blob:, Comlink); page-thread fallback
Storage	OPFS → IndexedDB → memory; model in Cache Storage; signed-in pages: memory only
Highlight	CSS Custom Highlight API, outline fallback

Options (`OwnerConfig`)

Option	Default	Meaning
`modes`	all three	Modes visitors may pick
`defaults`	Fast, top 5, short	Starting settings

Option	Default	Meaning
`questions`	none	Fixed questions (max 3 per page), `path` exact or `"/docs/*"`
`suggestions` · `suggestionCount`	`"both"` · 6	`"owner"`, `"dynamic"` or `"off"` · 3–6
`allowPins`	true	Visitors pin their own questions
`cache`	true	Save vectors on the device (never on signed-in pages)
`root`, `exclude`	auto	What to read; selectors never to read
`transformersUrl`, `embeddingModel`	jsDelivr, multilingual-e5-small	Self-host if you prefer
`webllmUrl`, `webllmModel`	off · Qwen2.5-0.5B-Instruct (q4f16)	WebLLM for Best mode
`webmcp`, `webmcpToken`	true · none	Read-only tools for browser agents; Chrome origin-trial token
`title`, `side`, `launcher`, `developer`		Panel look; "For developers" button

Mark anything private with `data-ask-private`. In Next.js, import the panel inside `useEffect` (it is browser-only).

Honest limits

- Fast mode matches words; paraphrases need Balanced mode. A question in another language than the page gets a clear hint (1.0.2): ask in the page's language, or turn on Balanced, whose model understands both.
- Chrome's built-in AI writes English, Japanese and Spanish; Telugu answers go through the Translator API when Chrome has that pair.
- Chrome's own model downloads are managed by Chrome; Ask page uses them only when already available.

6. YuktAI Toolkit `@yuktishaalaa/yuktai 6.0.3`

The library for apps you build, in Next.js, React and plain HTML. Includes the accessibility engine.

Part	What it does
📊 **Data grid**	Search, sort, filter, pages or one scrolling list, selection, CSV/Excel/JSON/PDF export, themes. 10 lakh rows with the Rust → WebAssembly core (TypeScript fallback). Parent and child rows. Card view on phones.
✨ **Grid assistant**	"Hyderabad orders above 1 lakh, biggest first" becomes checked steps (filter, filter, sort) shown as removable chips, with Undo and Reset. English and Telugu.
♿ **Accessibility widget**	Contrast, text size, dyslexia font, colour-blind filters, smart dark mode (6.0.3), WCAG audit, accessibility copilot.
💬 **Ask this page**	Answers from your page content.
🎨 **163 icons**	Any size and colour, safe in Server Components (`/icons`).
🧩 **YuktAI UI**	80+ accessible components with the names Material code already uses: change one import line, drop MUI and emotion. 44px targets, 3px focus ring, real `<dialog>`, arrow keys in Tabs and Menus.
🌐 **WebMCP**	Browser AI agents can search, count, filter and sort the grid through the same tools as the built-in assistant (`webmcp` prop).
🛠 **Page tools**	QA report, form review, design review, page insights (chapter 8).

```
<YuktaiGrid
  data={orders} columns={columns} rowKey="id"
  label="Orders"                // the table's name for screen readers
  ai                            // ✨ assistant button; Chrome's built-in AI plans requests
  getChildren={(o) => o.items}  // ▶ parent and child rows
  webmcp                        // tools for browser agents
/>
```

Library entry points

`@yuktishaalaa/yuktai` (main), `/icons`, `/experience` (loading screen, preferences, reveal), `/head` (no React: safe in a Next.js `<head>`), `/ui` (components). Plain HTML: `window.yuktai` from the CDN build, tested with jQuery 1.12.4, 2.2.4, 3.7.1 and 4.0.0.

7. Starters, CLI and generated apps

Command	
`npm create yuktai@latest my-app`	Create an app (asks a few questions). The usual way to start.
`npx yuktai@latest init my-app --yes`	The same with the `yuktai` command and the defaults
`npx yuktai@latest init simple --stack next --features none --yes`	Smallest app: Next.js only, no Rust, no Python

`init` options

Option	Values	Default
`--stack`	`next-rust` · `next`	`next-rust`
`--template`	`docs` · `starter`	`docs`
`--features`	`grid,page-tools,icons,backend` · `none`	all
`--host`	`none` · `netlify` · `github-pages`	`none`
`--accent`	`blue` · `violet` · `teal` · `orange`	`blue`
`--title`, `--no-install`, `--no-git`, `--yes`		

Inside a generated app

`npm run dev`	Builds the Rust crate, starts the Python API on a free port (if `backend/` exists), starts `next dev`. Ctrl + C stops all three.
`npm run build`	Static files in `out/`
`npm run dev:api` · `npm run test:api`	Python API only (http://localhost:8000/docs) · API tests (pytest)
`npm run test:e2e`	Browser tests (Playwright)
`npm run agent:test`	Testing Agent: runs every check and explains what failed
`npm run assistant`	YuktAI Assistant (developer agent): `add page contact`, `add api products`

Templates

`base` (Next.js app, menus, accessibility, PWA) · `feature-grid` (a /grid/ page with 25 lakh rows) · `feature-backend` (FastAPI, LangGraph agents, PostgreSQL files, Playwright tests) · `feature-icons`, `feature-page-tools` · `stack-next-rust` (a Rust crate built to WebAssembly by `yuktai dev`) · `host-netlify`, `host-github-pages`, `host-vercel`.

Python API and agents (optional)

- `backend/app/main.py` : FastAPI; new files in `routes/` load by themselves.
- `backend/app/agents/` : LangGraph agents: Grid Agent, Testing Agent, YuktAI Assistant.
- `backend/app/rag.py` : answers from `backend/knowledge/*.md` (BM25 retrieval).

- With a Groq key in `backend/.env` the agents use a language model (default `llama-3.3-70b-versatile`); without it they use rules ("free mode"). `YUKTAI_ENV=production` turns the developer agents off.

8. AI concepts used in YuktAI

YuktAI uses AI where it helps people, runs it on the device whenever possible, and keeps a working path without any model. This chapter lists every AI concept in the code, where it lives and what happens when it is not available.

8.1 Retrieval and search (no model needed)

Concept	Where	How YuktAI uses it
RAG (retrieval-augmented generation)	ask-page; Python `rag.py`	Find the best passages first, then answer only from them, and show the source. Without a model the answer is the passage itself (extractive).
Chunking into passages	ask-page reader	The live page is split into passages that keep their section heading, so the answer can cite and highlight its place.
Tokenization for Indian scripts	Rust core + TS twin	Unicode-aware words; Telugu and Devanagari combining marks stay together; compound words are joined; prefix matching.
BM25 keyword ranking	Rust `ask-core`; Python RAG	Classic relevance ranking with idf weights; the Fast mode engine.
Stop words / question words	`questionWords.ts`	Words that only make a question (ఎన్ని, అంటే, कितने) are removed before keyword search.
Embeddings (meaning vectors)	Transformers.js + multilingual-e5-small	Turns passages and questions into vectors in ~100 languages, with the model's `query:` / `passage:` prefixes; WebGPU when available, else WASM.
Vector quantization (int8)	Rust core	Vectors stored as 8-bit integers with a scale per passage: 4× smaller, fast dot products.
Cosine similarity	Rust core	Meaning match between question and passage.
Hybrid search + RRF (Reciprocal Rank Fusion, k = 60)	Rust core	Keyword and meaning rankings are merged, so both exact words and paraphrases are found.
Confidence and refusal	ask-page	idf-weighted coverage and cosine decide if the page really answers; otherwise "not on this page" instead of a guess (hallucination guard).
Evaluation	ask-page eval; grid evals	Ask: 53 questions over 7 pages; Fast mode recall@5 95.7%, top-1 82.6%, correct refusals 100% (from the package's eval report). Grid: `runGridEvals()` scores a planner on 50 English and Telugu requests.

8.2 Language models and generation

Concept	Where	How YuktAI uses it
On-device LLM (Gemini Nano, Prompt API)	ask-page `chromeAI.ts`; grid assistant	Writes answers from the retrieved passages, streamed. Used only when Chrome says the model is already available: YuktAI never starts Chrome's multi-GB download.

Concept	Where	How YuktAI uses it
Small LLM in the browser (WebLLM, WebGPU)	ask-page Best mode	`Qwen2.5-0.5B-Instruct` (4-bit). Off unless the site owner turns it on; downloads once after the visitor agrees.
Prompt engineering	`prompt.ts`	Page excerpts go inside delimiters with a random nonce; the system prompt says they are data only; the model must answer `NOT_IN_PAGE` when the excerpts do not hold the answer.
Prompt-injection defence	ask-page, grid, WebMCP	Page and row text is untrusted data ("ignore previous instructions" has no power); tools are read-only; results are marked `untrustedContentHint`.
Structured output + tool planning	Grid assistant	The model must answer in a JSON schema; each step is checked against the real tools and columns; at most 5 steps; downloads ask Yes/No first.
Summarization	Summarizer API	Page summaries on the device.
Machine translation	Translator API	Answers into Telugu/Hindi when Chrome has the language pair (Prompt API writes English, Japanese, Spanish).
Plain-language rewriting	Rewriter API (Toolkit)	Rewrites hard text into simpler words.
Language detection	a11y; Chrome LanguageDetector	Script-based detection in every browser; Chrome's on-device detector refines shared scripts.
Cloud LLM (optional, owner's key)	Generated app's Python API	Groq (`llama-3.3-70b-versatile` by default) for the developer agents; without a key, rules ("free mode"). Only the question and chosen passages are sent.

8.3 Agents and agentic web

Concept	Where	How YuktAI uses it
AI agents with tools	Python `agents/` (LangGraph)	Grid Agent (count, total, average, top, by city), Testing Agent (runs checks, explains failures), YuktAI Assistant (adds pages and API endpoints).
Agent graph / workflow	`@yuktishaalaa/ask-page/langgraph`	The five Ask steps as a LangGraph.js `StateGraph` (retrieve → check confidence → write answer → cite sources) plus a LangChain retriever. No tracing is sent anywhere.
WebMCP (tools for browser agents)	ask-page; grid	Read-only tools registered with `document.modelContext`: `ask_page`, `get_page_outline`, `get_suggested_questions`; grid tools (search, count, filter, sort, highlight…). Agents can read, never click, type or submit.
Task planning without a model	`taskRunner.ts`, `gridQuery.ts`	One instruction becomes ordered steps (filter, sort, count, summarise, anomalies) by rules; "overdue customers with balance over ₹10,000" becomes real filters.
Copilots	`a11yCopilot.ts`, `insights.ts`	Accessibility copilot: scans, explains, applies safe fixes, rescans on change. BI copilot: group, aggregate, compare periods, spot trends in plain English.

Concept	Where	How YuktAI uses it
Review agents	`qaAudit.ts`, `formReview.ts`, `designReview.ts`	QA pass with WCAG references; form flow review for seniors; design review from computed styles. Heuristics in the browser; no screenshots leave the device.
Suggested questions	ask-page `questions/engine.ts`	Owner questions, pinned questions, rules for the page type; each checked against the page first: no source, no suggestion.

8.4 Speech and voice

Text-to-speech	Web Speech `speechSynthesis`: read aloud with a voice per language part.
Speech recognition / voice commands	`voiceA11yCommands.ts`: "bigger text", "high contrast", next/previous grid row, by voice or typed.

8.5 Where the AI runs

Runs	Features	Without it
No model at all	BM25 search, quoted answers, page insights, task planner, QA/form/design reviews, copilots, language detection by script	Always available
On the device, downloaded once with consent	Embeddings (multilingual-e5-small), WebLLM	Fast mode
On the device, built into Chrome	Prompt API, Summarizer, Translator, Rewriter, LanguageDetector	Quoted answers; honest "search" label
In the cloud, only with the owner's key	Groq for the generated app's Python agents	Rule-based "free mode"

9. YuktAI లో వాడిన AI concepts (సారాంశం)

- **RAG:** ముందు page లో సరైన భాగాలు వెతికి, వాటి నుంచే జవాబు ఇస్తుంది, మూలాన్ని highlight చేస్తుంది. Model లేకపోయినా ఆ భాగాన్నే జవాబుగా చూపిస్తుంది.
- **BM25 keyword search (Rust → WebAssembly):** Fast mode. తెలుగు అక్షరాలను సరిగ్గా విడదీసే tokenizer. "ఎన్ని", "అంటే" లాంటి ప్రశ్న పదాలను తీసేస్తుంది.
- **Embeddings (multilingual-e5-small):** అర్థం ఆధారంగా వెతకడం, సుమారు 100 భాషల్లో. Visitor ఒప్పుకున్నాకే ఒక్కసారి download అవుతుంది.
- **Hybrid search + RRF:** పదాలు, అర్థం రెండు ఫలితాలను కలుపుతుంది. Vectors ని int8 గా చిన్నవి చేస్తుంది.
- **Confidence check:** page లో జవాబు లేకపోతే ఊహించకుండా "ఈ page లో లేదు" అని నిజాయితీగా చెప్తుంది.
- **On-device LLM:** Chrome లోని Gemini Nano (Prompt API). Chrome దగ్గర ఇప్పటికే ఉంటేనే వాడుతుంది. Best mode లో WebLLM (Qwen2.5-0.5B), site owner on చేస్తేనే.
- **Prompt injection రక్షణ:** page text ని data గానే చూస్తుంది. Tools చదవడానికి మాత్రమే.
- **Structured output:** grid assistant లో model JSON schema లోనే జవాబు ఇవ్వాలి. ప్రతి అడుగును నిజమైన tools తో సరిచూస్తుంది.
- **Summarizer, Translator, Rewriter, LanguageDetector:** Chrome built-in AI APIs.
- **Agents:** Python లో LangGraph agents (Grid, Testing, YuktAI Assistant). Groq key ఉంటే LLM, లేకపోతే rules తో "free mode". Ask flow LangGraph.js StateGraph గా కూడా ఉంది.
- **WebMCP:** browser లోని AI agents కి చదవడానికి మాత్రమే tools: `ask_page`, `get_page_outline`, `get_suggested_questions`, grid tools.
- **Copilots, review agents:** accessibility copilot, BI copilot, QA / form / design review. Model లేకుండానే browser లో నడుస్తాయి.
- **Voice:** Web Speech తో చదివి వినిపించడం (ప్రతి భాగానికి సరైన voice), voice commands.
- **Evaluation:** Ask కి 53 ప్రశ్నల eval (Fast mode: recall@5 95.7%), grid కి 50 English, తెలుగు requests.

> మూల సూత్రం: **AI ముందు device లోనే.** AI లేకపోయినా ప్రతి feature పనిచేస్తుంది, ఏం చేస్తోందో నిజాయితీగా చెప్తుంది. Visitor data బయటికి వెళ్ళదు.

10. Privacy and safety

Never read or sent

- What a visitor types: input, textarea and select values, rich editors.
- Passwords, one-time codes, card numbers (inputs are skipped whole).
- Hidden text, scripts, styles, templates; anything marked `data-ask-private` / `data-yuktai-private`; YuktAI's own panels.

Stored only on the device

- Visitor preferences (text size, theme, dark mode, panel language) in localStorage.
- Ask page vectors (OPFS → IndexedDB), never on signed-in or account pages; the model files in Cache Storage.

Opt-in only

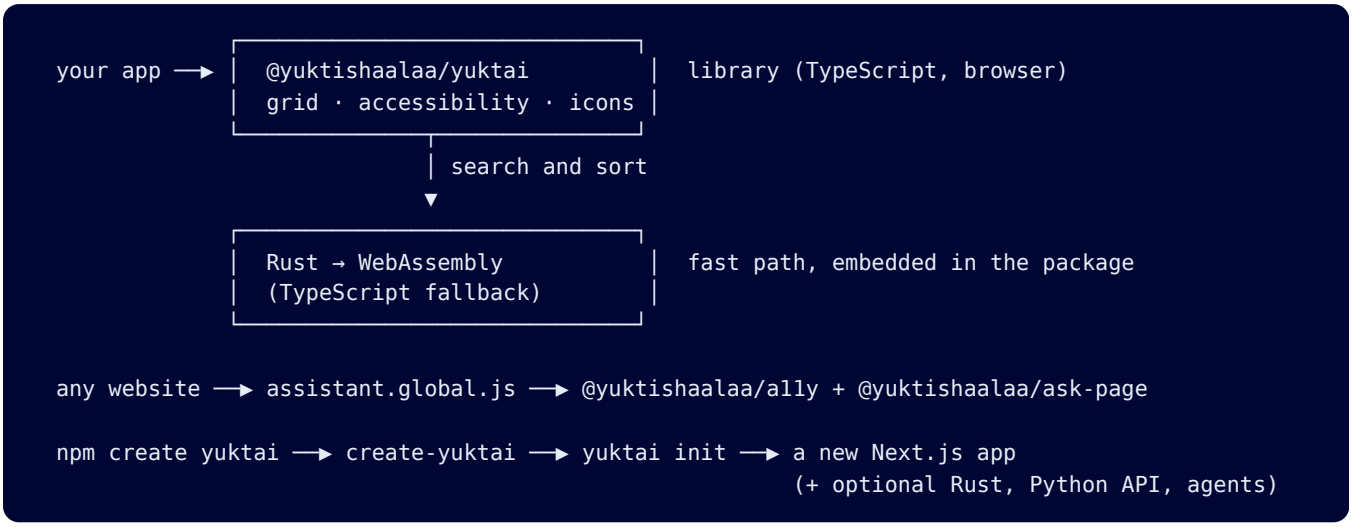
- Model downloads (Balanced/Best) ask the visitor first.
- Groq in the generated app's Python API needs the owner's key; only the question and chosen passages are sent.
- The browser extension runs on a page only after a click, or on a site switched to "Always on" (the browser asks for that site).

Safety against misuse

- Page text is untrusted: delimited with a nonce, "data only" in the system prompt, `NOT_IN_PAGE` when unsure.
- Agent tools are read-only; grid downloads ask Yes/No; at most 5 planned steps, each validated.
- Strict Content Security Policy pages work: when WebAssembly is blocked, the TypeScript search twin is used.

> **Honest claims.** YuktAI makes common problems hard and reports what a person must check; it does not certify a site as "WCAG compliant". Compliance with laws (for example, India's RPwD Act and DPDP Act) stays the site owner's responsibility.

11. Architecture and repository



Folder	Contents
`src/`	Toolkit library: grid, core (renderer, AI), agentic, experience, icons, ui, next, react
`packages/a11y`, `packages/assistant`, `packages/ask-page`	The engines and the Assistant (each with its own tests, README, CHANGELOG)
`packages/assistant/extension`	MV3 browser extension (activeTab; content script in the page's world)
`packages/create-yuktai`, `packages/yuktai-cli`	Starter and CLI with templates
`rust/`	`ask-core`, `grid-core`, `rag-core` crates → WebAssembly
`vanilla/`, `ask-page/`, `examples/`	Plain HTML build and demos, jQuery example
`site/`	Docs site: Next.js 16, App Router, static export to GitHub Pages under `/yuktai`
`docs/modules/`	One page per AI/agent module

Build tools

TypeScript built with tsup (ESM, CJS, IIFE for the script tag). Rust stable, target `wasm32-unknown-unknown`, embedded in the package so users install nothing extra. The Toolkit bundles the a11y engine from source. Vitest with happy-dom for unit tests; Playwright + axe-core for browser tests; Oxlint and Biome.

Docs site (Next.js 16)

App Router with a `(guides)` route group, Server Components for content and Client Components only for interactive parts (header, theme, Ask box, count-up animations). Metadata per page, share image (`opengraph-image.png`), `sitemap.xml`. The GitHub Action builds this commit's packages, the site, runs every check and deploys to GitHub Pages.

12. Testing and quality

Suite	Count	Result at f133b30
Toolkit unit tests (Vitest)	184	**pass**
a11y unit tests	52	**pass**
ask-page unit tests (+ eval)	64	**pass**
Assistant unit tests	15	**pass**
All TypeScript unit tests	**315**	**pass**
Plain HTML end-to-end (Chromium) + axe	37	**pass · 0 axe violations**
jQuery 1.12.4, 2.2.4, 3.7.1, 4.0.0 with YuktAI 6.0.3 + Ask 1.0.2	4	**pass**
Website: 18 pages + 2 demos, links, axe serious/critical	68 links	**0 broken · 0 axe**
Website deep scan: 36 views (desktop light + phone dark), axe incl. best practice, overflow, headings, ids	36	**clean**
Rust crates (cargo test)	13	run in the GitHub Action

Not yet tested

Real screen readers (NVDA, JAWS, TalkBack, VoiceOver), Firefox and Safari by hand, a real Android phone, Telugu read aloud with a real Telugu voice.

13. Releasing: GitHub, npm, website

1. Code to GitHub (Windows PowerShell)

```
cd C:\Users\vanis\Github\yuktai
npm install
npm run build:all
npm test
git add -A
git commit -m "YuktAI 6.0.3: ..."
git push origin main
```

2. npm (staged publishing)

npm stages every new version; it becomes public only after approval. Publish engines first, then the products that depend on them.

```
cd packages\a11y
npm publish --access public          # then approve a11y on npmjs.com
cd ..\assistant
npm install
npm publish --access public          # then approve assistant
cd ..\..
npm publish --access public          # the Toolkit; then approve
```

- Approve on **npmjs.com → Staged Packages** (2FA code from your authenticator), or with `npm stage list` / `npm stage approve <id>` (needs npm 11.15+ and Node 22.14+).
- **E409 "Cannot publish over previously staged version"** means that version is already staged: approve it, do not publish again.
- Check: `npm view @yuktishaalaa/a11y version`.

3. Website

A push to `main` runs the "Site" GitHub Action: build, tests, site checks, deploy to GitHub Pages. If the live site does not change, open **Actions → Site** for the error, and check **Settings → Pages → Source = GitHub Actions**.

Commit rule

Commits go to `main` (no branches). Commits made by Claude carry author and committer `Claude <noreply@anthropic.com>`.

14. Release history

6.0.3 / Assistant 6.0.4 — 14 Nov 2026

- **Website** (full scan of 18 pages): loading screen 15 s → 1.5 s; home shows the six packages as one family with a self-drawing diagram and count-up numbers (only when motion is allowed); new hero text; Assistant guide code no longer overflows (was 798 px on phones); jQuery and Next.js guides install current versions; code wraps and wide tables are keyboard-scrollable; sticky header no longer covers `#section` targets on phones; share image and sitemap.
- **Toolkit 6.0.3:** ♿ widget uses smart dark mode; grid card view no longer puts `aria-selected` on list items (axe critical); demo version stamped from package.json.
- **a11y 6.0.3:** dark mode for CSS icons and gradient text; calmer on changing pages.
- **Assistant 6.0.4:** "What to check" list; translated tab name.

Assistant 6.0.3 — 14 Nov 2026

Config check with console warnings; new `panelLang` and `offset`; Ask opens on the same side; docs show every option; panel stays on screen when moved up.

6.0.2 / Ask 1.0.2 — 14 Nov 2026

Smart dark mode (each colour on its own, AA/AAA contrast kept; photos, emoji, brand colours stay right). Ask: clear hint when the question and page are in different languages.

6.0.1 / Ask 1.0.1 — 14 Nov 2026

Read aloud skips emoji and symbols, reads only visible main content, keeps short English words in Telugu sentences; text size via `zoom`; no dark mode on dark pages; checks for symbol-only buttons and broken skip links; SPA and shadow-root language. Assistant: site title and colour, draggable slider, lasting messages, panel language picker, High Contrast, Go to main content, Alt+Shift+Y, one Ask button. Ask: Telugu/Hindi question words, focus returns on close, honest "search" label.

6.0.0

The **YuktAI Assistant** and the new accessibility engine **@yuktishaalaa/a11y**. Breaking: the engine no longer invents labels or states ("do no harm"); `<html lang>` is detected from the page text. Browser extension and the Assistant page on the docs site. Telugu IME Enter fix; grid arrow keys.

5.x (summary)

5.7.0: agentic grid: Chrome's built-in AI plans plain-language requests with structured JSON output and tool calling (up to 5 checked steps, one corrected re-plan), shown as removable chips with Undo and Reset; new tools `download` (asks Yes/No), `reset_view`, `undo`, `expand`. **5.6:** YuktAI UI, 80+ accessible components with Material's names and no runtime dependencies. **5.5:** WebMCP results as JSON strings with `readOnlyHint` / `untrustedContentHint`; loading screen, preferences (language and animations) and Reveal. **5.4:** "Show all": 10 lakh rows in one virtual scrolling list; password and typed values never read. **5.3:** page buttons, page numbers and "Go to page".

15. Known limits and roadmap

Open items

- Ask: answers from a table row do not show the row label yet.
- React sites: the 🔊 buttons added next to headings can clash with hydration.
- Extension: settings are saved per site, not once for all sites.
- Dark mode does not reach inside shadow DOM; dark transparent PNG logos may be hard to see on a dark page.
- Ask Fast mode cannot match a question in another language than the page (Balanced can).
- Sanskrit: shares the Devanagari script with Hindi, so it must come from `<html lang="sa">`; files to change are listed in the project notes.

Roadmap (from ROADMAP.md)

- **Now:** `npm create yuktai@latest` as the main start; a small default app; one `packages/` folder with workspaces; lint warnings.
- **Next:** translate every menu and button; more Indian languages (all 22 scheduled languages, including Sanskrit); `yuktai add` commands; agent-to-agent (A2A) between YuktAI agents.
- **Later:** columnar grid storage in Rust; more agent logic in Rust; Angular, Vue and .NET; a Python package.

Ideas under discussion

- Agentic accessibility: a live narrator that says what changed after a click; fix → verify → undo loops; accessibility as WebMCP tools.
- **Yukt**: a Python-like language of its own for the web, where accessibility and privacy are part of the syntax (`screen` , `engine` , `kind` , `means` , `says` , `private`), compiled to a Next.js app.

16. Glossary

WCAG 2.2 A / AA / AAA	The W3C accessibility guidelines and their levels; AA is the usual legal minimum (contrast 4.5:1), AAA stricter (7:1).
ARIA	Attributes that tell assistive technology what an element is and its state.
Shadow DOM / Web component	An element with its own private DOM and styles (`<yuktai-assistant>` , `<yuktai-ask>`).
WebAssembly (WASM)	Fast compiled code in the browser; YuktAI's Rust engines run as WASM.
RAG	Retrieval-augmented generation: find relevant passages first, then answer only from them.
BM25	Keyword relevance ranking.
Embedding	A vector that holds the meaning of a text; close vectors mean similar meaning.
RRF	Reciprocal Rank Fusion: merges two rankings by position.
Quantization (int8)	Storing numbers in 8 bits to save memory and time.
LLM / on-device model	A language model; on-device means it runs on the visitor's computer (Gemini Nano, WebLLM).
Prompt injection	Text that tries to give orders to a model; YuktAI treats page text as data.
Agent / tool	A program that uses a model to plan and call tools; a tool is one safe action (count rows, read the outline).
LangGraph	A library for agent workflows as graphs of steps.
WebMCP	A browser standard proposal that lets pages offer tools to AI agents (`document.modelContext`).
Staged publishing	npm keeps a new version private until the owner approves it.

© Sandeep Miriyala — Yuktishaalaa AI Lab. GPL-3.0-only. Docs: sandeep-miriyala.github.io/yuktai · Code: github.com/sandeep-miriyala/yuktai · npm: npmjs.com/~yuktishaalaa